



maiora premunt / gjëra të mëdha na presin



KODI: A.IV.3.3 / ESE (ARGUMENTUESE, AKADEMIKE, OSE KËRKIMORE)

UNIVERSITETI EUROPIAN I TIRANËS

Pedagogu: Dr. Ardiana Topi, Msc. Jurgen Kruja

Lënda: Zhvillime Webi: Aplikime dhe Programim

Tipologjia: T2

Programi i studimit: MSH2IE

Viti Akademik: 2025-2026

Tema: Zhvillimi i një aplikacioni web full-stack me MERN që simulon një sistem menaxhimi real, me funksionalitete bazë të aksesit të të dhënave dhe autentifikimit. (Menaxhimi i Shpenzimeve Ditore)

Denis Ajazi

TIRANË, QERSHOR, 2026

I. HYRJE

Menaxhimi i financave personale është një nga aspektet më të rëndësishme të organizimit të përditshëm. Shumë individë hasin vështirësi në monitorimin e shpenzimeve të tyre, gjë që mund të çojë në keqmenaxhim të buxhetit dhe mungesë kontrolli mbi financat personale.

Qëllimi i këtij projekti është zhvillimi i një aplikacioni web të quajtur “Budget Savings App”, i cili mundëson regjistrimin, ruajtjen, përditësimin dhe fshirjen e shpenzimeve personale. Aplikacioni ofron një ndërfaqe moderne dhe intuitive për përdoruesit, si dhe një sistem të sigurt autentifikimi përmes JSON Web Token (JWT).

Projekti është ndërtuar duke përdorur arkitekturën MERN (MongoDB, Express.js, React.js dhe Node.js), duke kombinuar teknologjitë moderne të zhvillimit web për krijimin e një aplikacioni të plotë Full Stack.

Github: <https://github.com/denisajaziu/budget-savings-app>

II. ZHVILLIM

3. Zhvillimi i Projektit

3.1 Analiza e Kërkesave

Qëllimi kryesor i këtij projekti ishte ndërtimi i një aplikacioni web për menaxhimin e shpenzimeve personale. Gjatë analizës së kërkesave u identifikua nevoja për një sistem që t'i lejojë përdoruesit të regjistrohen, të identifikohen në mënyrë të sigurt dhe të menaxhojnë shpenzimet e tyre personale. Sistemi duhet të ofrojë ruajtje të të dhënave në databazë, mundësi për shtimin, përditësimin dhe fshirjen e shpenzimeve, si dhe paraqitjen e statistikave financiare në një formë të kuptueshme.

Për të realizuar këto kërkesa u vendos përdorimi i arkitekturës MERN (MongoDB, Express.js, React.js dhe Node.js). Kjo arkitekturë u zgjodh sepse ofron një ndarje të qartë ndërmjet frontend-it dhe backend-it, mundëson zhvillim modern të aplikacioneve web dhe siguron shkallëzueshmëri për zgjerime të ardhshme.

Gjatë fazës së planifikimit u vendos që aplikacioni të përfshijë funksionalitetet e mëposhtme:

- Regjistrimin e përdoruesve të rinj.
- Identifikimin e përdoruesve ekzistues.
- Ruajtjen e sigurt të fjalëkalimeve.

- Menaxhimin e shpenzimeve personale.
- Analizimin e të dhënave financiare.
- Paraqitjen e statistikave në dashboard.
- Mbrojtjen e route-ve private.
- Ruajtjen e të dhënave në MongoDB Atlas.

Këto funksionalitete përbëjnë bazën e një sistemi modern për menaxhimin e financave personale.

3.2 Arkitektura e Aplikacionit

Aplikacioni është ndërtuar sipas modelit Client – Server.

Frontend-i është përgjegjës për ndërfaqen grafike dhe ndërveprimin me përdoruesin. Ai është zhvilluar me React dhe komunikon me backend-in nëpërmjet kërkesave HTTP.

Backend-i është zhvilluar me Node.js dhe Express.js dhe është përgjegjës për:

- Autentifikimin e përdoruesve.
- Menaxhimin e route-ve.
- Kryerjen e operacioneve CRUD.
- Komunikimin me databazën.
- Mbrojtjen e të dhënave.

Databaza MongoDB Atlas përdoret për ruajtjen e të dhënave të përdoruesve dhe shpenzimeve.

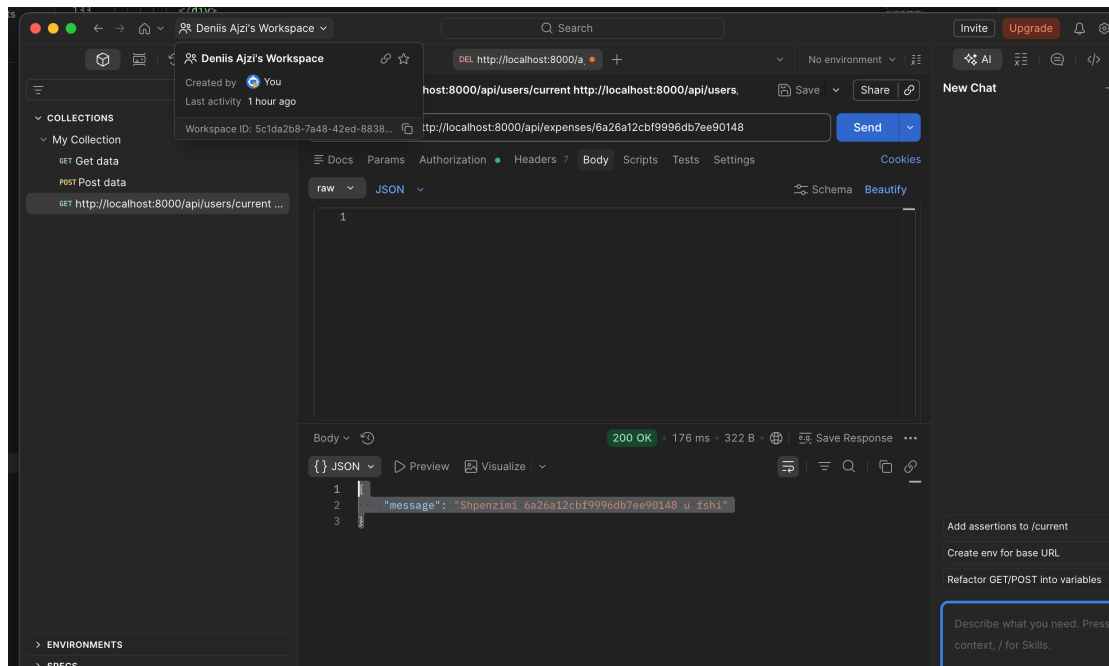
Arkitektura e përgjithshme funksionon sipas skemës:

Përdoruesi → React Frontend → Express API → MongoDB Atlas

Kur përdoruesi kryen një veprim në ndërfaqe, frontend-i dërgon një kërkesë te backend-i. Backend-i e përpunon këtë kërkesë dhe komunikon me databazën. Rezultati kthehet përsëri te frontend-i dhe paraqitet në ekran.

Kjo arkitekturë mundëson ndarje të qartë të përgjegjësiave dhe e bën aplikacionin më të organizuar.

3.3 Zhvillimi i Backend



Backend-i përbën pjesën kryesore logjike të sistemit. Ai u zhvillua duke përdorur Node.js dhe Express.js.

Fillimisht u krijua projekti backend dhe u instaluan paketat e nevojshme si:

- Express
- Mongoose
- Dotenv
- Cors
- Bcryptjs
- Jsonwebtoken
- Express Async Handler
- Nodemon

Express u përdor për ndërtimin e API-së, ndërsa Mongoose për komunikimin me MongoDB.

Për organizimin e kodit u përdor struktura:

- Config

- Models
- Controllers
- Routes
- Middleware

Kjo strukturë ndan logjikën në mënyrë të qartë dhe e bën projektin më të mirëmbajtshëm.

3.3.1 Konfigurimi i Serverit

Serveri u konfigurua në skedarin server.js.

Në këtë skedar:

- U inicializua Express.
- U konfigurua dotenv.
- U aktivizua CORS.
- U shtua mbështetja për JSON.
- U krijuan route-t kryesore.
- U lidh middleware për gabimet.

Serveri ekzekutohet në portën 8000 dhe komunikon me frontend-in nëpërmjet REST API.

Përdorimi i Express mundësoi krijimin e endpoint-eve të nevojshme për aplikacionin.

3.3.2 Konfigurimi i MongoDB Atlas

Për ruajtjen e të dhënave u përdor MongoDB Atlas.

MongoDB Atlas është një platformë cloud që mundëson menaxhimin e databazave MongoDB pa pasur nevojë për instalim lokal.

Në projekt u krijua një cluster i ri në MongoDB Atlas dhe u gjenerua një connection string për lidhjen me backend-in.

Ky connection string u ruajt në skedarin .env për arsye sigurie.

Për realizimin e lidhjes u përdor Mongoose.

Mongoose krijon një shtresë abstraksioni mbi MongoDB dhe lehtëson krijimin e modeleve dhe ekzekutimin e operacioneve CRUD.

Pasi lidhja u realizua me sukses, aplikacioni ishte në gjendje të ruante dhe lexonte të dhëna nga databaza.

3.3.3 Modeli User

Modeli User u krijua për të përfaqësuar përdoruesit e sistemit.

Ky model përmban fushat:

- name
- email
- password

Email-i është unik për çdo përdorues.

Kjo siguron që dy përdorues të mos regjistrohen me të njëjtën adresë email.

Modeli përdoret gjatë:

- Regjistrimit
- Identifikimit
- Verifikimit të përdoruesit

Përdorimi i modeleve ndihmon në standardizimin e strukturës së të dhënave në databazë.

3.3.4 Modeli Expense

Modeli Expense përfaqëson shpenzimet e përdoruesve.

Fushat kryesore janë:

- description
- amount
- category
- date
- user

Fusha user përmban referencën e përdoruesit që zotëron shpenzimin.

Kjo bën të mundur që çdo përdorues të shohë vetëm të dhënat e tij.

Kategoritë e përdorura janë:

- Ushqim
- Transport
- Fatura
- Argëtim
- Tjetër

Kjo strukturë lejon analizimin e shpenzimeve sipas kategorive.

3.3.5 Regjistrimi i Përdoruesit

Regjistrimi realizohet përmes endpoint-it:

POST /api/users

Përdoruesi dërgon:

- Emrin
- Email-in
- Fjalëkalimin

Sistemi kontrollon nëse ekziston tashmë një përdorues me të njëjtin email.

Nëse përdoruesi nuk ekziston, fjalëkalimi hashohet dhe përdoruesi ruhet në databazë.

Pas regjistrimit gjenerohet automatikisht një JWT Token.

Ky token përdoret për autentifikimin e mëvonshëm.

3.3.6 Identifikimi i Përdoruesit

Identifikimi realizohet përmes endpoint-it:

POST /api/users/login

Sistemi kërkon:

- Email
- Fjalëkalim

Pasi përdoruesi gjendet në databazë, fjalëkalimi krahasohet me hash-in e ruajtur.

Nëse të dhënat janë të sakta, backend-i kthen:

- ID
- Emrin
- Email-in
- JWT Token

Në rast gabimi shfaqet një mesazh përkatës.

3.3.7 Siguria me bcrypt

Për ruajtjen e sigurt të fjalëkalimeve u përdor biblioteka bcryptjs.

Hashimi është procesi i transformimit të fjalëkalimit në një formë të pakthyeshme.

Kjo do të thotë se fjalëkalimi nuk ruhet kurrë në formën e tij reale.

Edhe në rast komprometimi të databazës, fjalëkalimet nuk mund të lexohen lehtësisht.

Përdorimi i bcrypt konsiderohet praktikë standarde në zhvillimin e aplikacioneve moderne.

3.3.8 Siguria me JWT

JWT përdoret për autentifikimin e përdoruesve.

Pas identifikimit, serveri gjeneron një token unik.

Ky token ruhet në frontend dhe dërgohet në çdo kërkesë të mbrojtur.

Backend-i verifikon token-in përpara se të lejojë aksesin në route-t private.

Kjo metodë shmang përdorimin e sesioneve tradicionale dhe ofron performancë më të mirë.

3.3.9 Auth Middleware

Auth Middleware është përgjegjës për mbrojtjen e route-ve private.

Ky middleware:

- Lexon token-in nga header-i Authorization.
- Verifikon vlefshmërinë e tij.
- Merr të dhënat e përdoruesit.
- Lejon ose ndalon aksesin.

Nëse token-i mungon ose është i pavlefshëm, sistemi kthen gabim 401 Unauthorized.

3.3.10 Operacionet CRUD

Aplikacioni implementon operacionet bazë CRUD.

Create përdoret për shtimin e shpenzimeve.

Read përdoret për marrjen e listës së shpenzimeve.

Update përdoret për përditësimin e shpenzimeve ekzistuese.

Delete përdoret për fshirjen e tyre.

Çdo operacion kontrollon pronësinë e të dhënave për të garantuar sigurinë.

3.4 Zhvillimi i Frontend

Frontend-i u ndërtua me React.js.

React u zgjodh sepse mundëson ndërtimin e ndërfaqeve dinamike përmes komponentëve të ripërdorshëm.

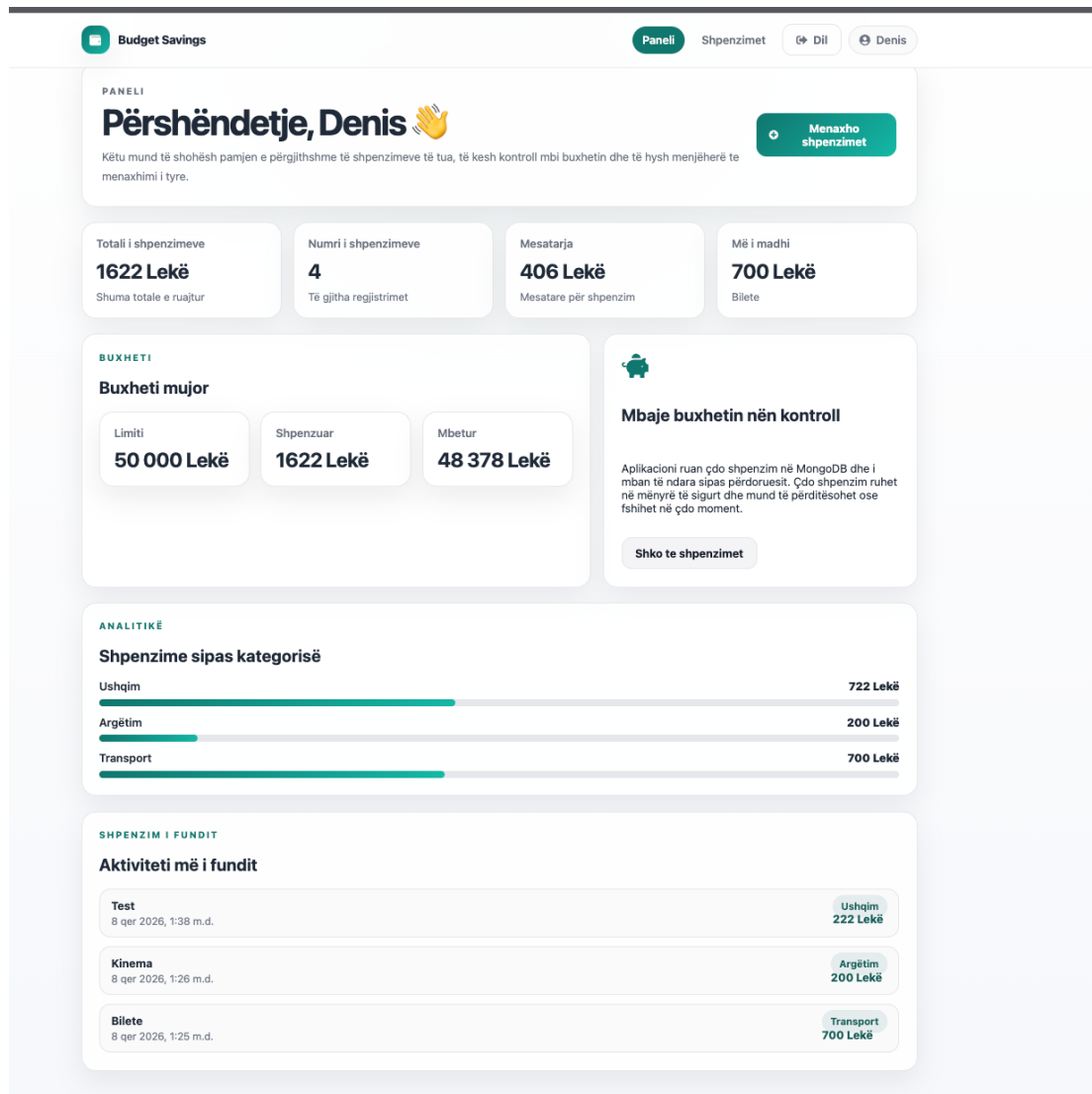
Për krijimin e projektit u përdor Vite, i cili ofron performancë më të mirë gjatë zhvillimit.

3.4.1 React Router

React Router u përdor për menaxhimin e navigimit.

U krijuan faqet:

- Hyrje



- Regjistrim
- Dashboard
- Shpenzimet

Navigimi realizohet pa rifreskim të faqes, duke përmirësuar përvojën e përdoruesit.

3.4.2 Redux Toolkit

Redux Toolkit u përdor për menaxhimin e state-it global.

Informacioni i përdoruesit ruhet në Redux Store dhe është i disponueshëm në të gjithë aplikacionin.

Kjo shmang kalimin e të dhënave nëpër komponentë dhe e bën aplikacionin më të organizuar.

3.4.3 RTK Query

RTK Query u përdor për komunikimin me backend-in.

Ai menaxhon:

- Dërgimin e kërkesave HTTP.
- Marrjen e përgjigjeve.
- Rifreskimin automatik të të dhënave.
- Menaxhimin e cache.

Kjo redukton ndjeshëm sasinë e kodit të nevojshëm.

3.4.4 Dashboard

Dashboard-i është faqja kryesore e aplikacionit.

Ai paraqet:

- Totalin e shpenzimeve.
- Numrin e shpenzimeve.
- Mesataren.
- Shpenzimin më të madh.
- Buxhetin mujor.
- Analitikën sipas kategorive.

Kjo i jep përdoruesit një pasqyrë të menjëhershme të gjendjes financiare.

3.4.5 Menaxhimi i Shpenzimeve

Faqja e shpenzimeve përmban formularin dhe listën e shpenzimeve.

Përdoruesi mund:

Budget Savings

Paneli Shpenzimet Dil Denis

MENAXHIMI

Shpenzimet e mia

Totali i filtruar: 1622 Lekë

Këtu mund të shtosh, përditësosh dhe fshish shpenzimet.

FORMULAR

Shto shpenzim

Përshkrimi Shuma

p.sh. Dreka p.sh. 500

Kategoria Data

Ushqim 08.06.2026

Rua

Totali

1622 Lekë

Shuma totale

Numri i shpenzimeve

4

Të gjitha regjistrimet

Mesatarja

406 Lekë

Mesatare për shpenzim

LISTA

Shpenzimet e regjistruara

Shto të re

8 qer 2026, 1:38 m.d.	Ushqim	222 Lekë	
8 qer 2026, 1:26 m.d.	Argëtim	200 Lekë	
8 qer 2026, 1:25 m.d.	Transport	700 Lekë	
8 qer 2026, 1:25 m.d.	Ushqim	500 Lekë	
Shpenzimi më i madh:		Bilete — 700 Lekë	

Budget Savings App
Punuar nga Denis Ajazi

- Të shtojë shpenzime.
- Të përditësojë shpenzime.
- Të fshijë shpenzime.
- Të shikojë historikun e tyre.

Ndryshimet reflektohen menjëherë në ndërfaqe.

3.4.6 Dizajni dhe Përvoja e Përdoruesit

Ndërfaqja është ndërtuar me fokus te thjeshtësia dhe përdorshmëria.

Janë përdorur:

- Karta statistikash.
- Ikona.
- Ngjyra neutrale.
- Layout responsive.

Kjo siguron funksionim korrekt si në kompjuter ashtu edhe në pajisje mobile.

3.5 Testimi i Aplikacionit

Për testimin e backend-it u përdor Postman.

U testuan me sukses:

- Register
- Login
- Current User
- Create Expense
- Read Expense
- Update Expense
- Delete Expense

Gjithashtu u testua komunikimi ndërmjet frontend-it dhe backend-it.

Rezultatet treguan se aplikacioni funksionon sipas specifikimeve dhe përmbush kërkesat e projektit.

III. PËRFUNDIM

Në përfundim, projekti **“Budget Savings App”** arriti të realizojë me sukses qëllimin kryesor për të cilin u zhvillua, duke ofruar një sistem të plotë për menaxhimin e

shpenzimeve personale. Gjatë zhvillimit të projektit u përdor arkitektura MERN (MongoDB, Express.js, React.js dhe Node.js), e cila mundësoi ndërtimin e një aplikacioni modern, të organizuar dhe të shkallëzueshëm.

Aplikacioni ofron funksionalitete të rëndësishme si regjistrimi i përdoruesve, identifikimi i sigurt në sistem, ruajtja e fjalëkalimeve në formë të hashuar, menaxhimi i shpenzimeve personale dhe paraqitja e statistikave financiare në kohë reale. Implementimi i JWT dhe middleware-ve të autorizimit siguroi mbrojtjen e route-ve private dhe aksesin vetëm për përdoruesit e autorizuar.

Për menaxhimin e state-it global u përdor Redux Toolkit, ndërsa RTK Query u përdor për komunikimin ndërmjet frontend-it dhe backend-it. Këto teknologji ndihmuan në organizimin më të mirë të aplikacionit dhe reduktuan kompleksitetin e kodit. Nga ana tjetër, MongoDB Atlas siguroi një zgjidhje moderne cloud për ruajtjen dhe menaxhimin e të dhënave.

Gjatë realizimit të projektit u fituan njohuri praktike mbi zhvillimin Full Stack, organizimin e kodit sipas strukturave profesionale, implementimin e autentifikimit dhe autorizimit, si dhe integrimin e teknologjive të ndryshme në një sistem të vetëm funksional. Projekti demonstroi rëndësinë e ndarjes së përgjegjësisë ndërmjet frontend-it dhe backend-it, si dhe përdorimin e praktikave moderne të zhvillimit web.

Rezultatet e testimeve treguan se aplikacioni funksionon sipas kërkesave të përcaktuara, duke realizuar me sukses të gjitha operacionet CRUD, autentifikimin e përdoruesve dhe komunikimin me databazën. Përveç funksionalitetit teknik, aplikacioni ofron edhe një ndërfaqe të thjeshtë, intuitive dhe të lehtë për përdorim.

Në të ardhmen, projekti mund të zgjerohet me funksionalitete shtesë si raportime mujore të avancuara, grafikë interaktivë, eksportim të të dhënave në formate të ndryshme, kategorizim më të detajuar të shpenzimeve, vendosje të objektivave financiare dhe integrim me sisteme të tjera financiare. Këto zgjerime do ta bënin aplikacionin edhe më të dobishëm për përdoruesit dhe më afër një produkti real të përdorshëm në praktikë.

Në përgjithësi, projekti “Budget Savings App” përmbush me sukses objektivat e vendosura dhe përfaqëson një zbatim praktik të njohurive të fituara gjatë lëndës së Zhvillimit të Aplikacioneve Web.

IV. BIBLIOGRAFIA

React Documentation – <https://react.dev>
 Redux Toolkit Documentation – <https://redux-toolkit.js.org>
 RTK Query Documentation – <https://redux-toolkit.js.org/rtk-query>
 Node.js Documentation – <https://nodejs.org>
 Express.js Documentation – <https://expressjs.com>
 MongoDB Documentation – <https://www.mongodb.com/docs>
 Mongoose Documentation – <https://mongoosejs.com>
 JSON Web Token – <https://jwt.io>
 bcryptjs Documentation – <https://www.npmjs.com/package/bcryptjs>
 Postman Documentation – <https://learning.postman.com>

Vlerësimi i pedagogut:	Pikët:
I. Parashtrimi i tezës së esesë:	
II. Argumentimi:	
III. Përfundimi:	



maiora premunt / gjëra të mëdha na presin



	Totali	
--	---------------	--